

QualityHR – Selenium Java Automation Framework for HR Web Application

1. Project Overview

This project focuses on building a Selenium-based test automation framework using Java to automate the OrangeHRM web application.

The framework follows the Page Object Model (POM) design pattern to achieve:

- High scalability
- Easy maintenance
- Reusability of components

It is designed to support functional testing, validation checks, and complete end-to-end workflows.

2. Objectives

The main goals of this framework include:

- Automating complete UI-based test scenarios
- Developing a structured and reusable automation framework
- Utilizing TestNG for test execution and lifecycle handling
- Managing dynamic web elements using explicit wait strategies
- Capturing screenshots automatically upon test failures using ITestListener
- Generating detailed HTML reports through ExtentReports
- Enabling configuration-based test execution

3. Application Under Test

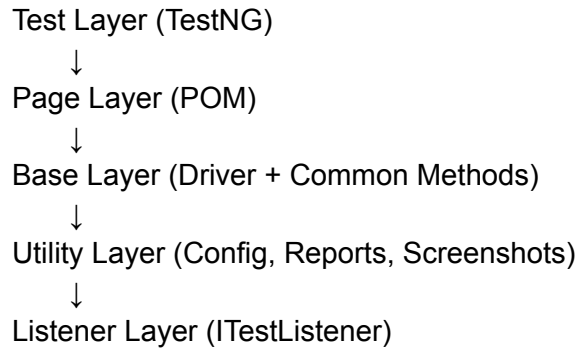
- **URL:** <https://opensource-demo.orangehrmlive.com>
- **Application Type:** HR Management System

Modules Covered

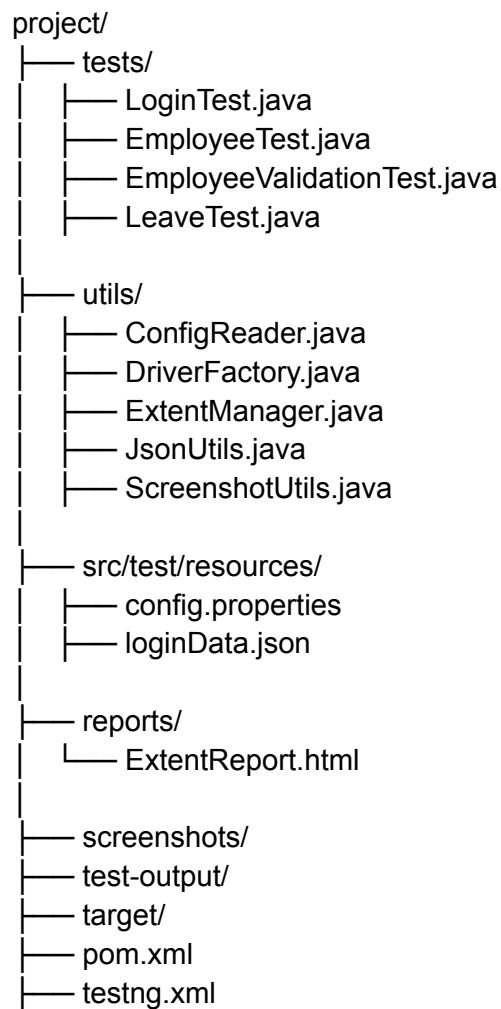
- Login
- PIM (Employee Management)
- Admin (User Management)
- Leave

4. Framework Architecture

The framework is structured using the Page Object Model (POM) approach:



5. Project Structure



6. Key Components

a. BasePage

Includes commonly used reusable methods such as:

- click()
- sendKeys()
- wait handling

It acts as the foundation for all page interactions.

b. Test Classes

- LoginTest
- EmployeeTest
- EmployeeValidationTest
- LeaveTest

Responsibilities:

- Define test cases
- Perform validations and assertions
- Invoke methods from page classes

c. Page Classes

- LoginPage
- DashboardPage
- EmployeePage
- AdminPage
- LeavePage

Responsibilities:

- Maintain element locators
- Perform UI interactions
- Encapsulate page-specific functionality

d. DriverFactory

- Responsible for WebDriver initialization
- Supports multiple browser configurations

e. ConfigReader

- Reads values from `config.properties`
- Eliminates the need for hardcoded data

f. ScreenshotUtils

- Captures screenshots during execution
- Stores them in the `/screenshots/` directory

g. ExtentManager

- Handles configuration of ExtentReports
- Generates structured HTML reports

h. TestListener (ITestListener)

- Monitors test execution lifecycle
- Captures screenshots on failures
- Logs results into reports

7. Configuration Management

All configurable values are stored in the `config.properties` file.

Example:

```
browser=chrome  
baseUrl=https://opensource-demo.orangehrmlive.com  
timeout=10  
admin.username=Admin  
admin.password=admin123
```

Ensures no hardcoded values are used

8. Wait Strategy

The framework uses **Explicit Waits (WebDriverWait)** to handle:

- Dynamic elements
- Loading delays
- Dropdown rendering

Avoids using `Thread.sleep` for better performance

9. Test Modules Covered

Employee Module

- Add Employee
- Verify employee in list
- Search employee
- View employee details

Validation Testing

- Mandatory field validation
- Incorrect date format validation
- Dropdown selection validation

Admin Module

- Create system user (ESS role)
- Validate user presence
- Delete user

Leave Module

- Apply leave
- Check leave status
- Validate invalid inputs

End-to-End Scenario

- Create Employee
- Create System User
- Verify User
- Delete User

10. Screenshot Capture

- Implemented using ITestListener
- Automatically triggered on test failures
- Saved under [/screenshots/](#)
- Embedded into Extent Reports

11. Reporting

- **Tool Used:** ExtentReports
- **Location:** `/reports/ExtentReport.html`

Report Includes:

- Test execution results
- Logs
- Error details
- Failure screenshots

12. Key Design Decisions

- Adopted POM for better maintainability
- Used configuration-driven design
- Implemented listener-based reporting
- Applied explicit wait strategies
- Eliminated hardcoding

13. Advantages

- ✓ Well-structured modular design
- ✓ Easy to maintain and scale
- ✓ Handles dynamic web elements effectively
- ✓ Reusable components
- ✓ Professional-level reporting
- ✓ Suitable for real-world scenarios

14. Conclusion

This framework represents a complete Selenium automation solution featuring:

- Clean and organized architecture
- Validation testing capabilities
- End-to-end scenario coverage
- Reporting with integrated screenshots

It is ideal for:

- Academic projects
- Interview demonstrations

- Industry-level automation implementations